NΛCO

Pronuncia-se "NACO"

OBJETIVO

- O NΛCO é um projeto de código aberto voltado para a área alimentícia. Assim como plataformas como o Ifood, o objetivo principal do NΛCO é a praticidade. Com poucos cliques, o usuário consegue realizar pedidos de comida de forma rápida e eficiente.
    - O NΛCO nasceu com o objetivo de ser muito mais do que um simples aplicativo. Para isso, é importante conhecer seu criador; meu nome é Gabriel, sou estudante de Ciências da Computação e apaixonado por tecnologia. Criei esse aplicativo com o objetivo de aplicar meus conhecimentos conforme minha progressão na carreira, além de contribuir com a comunidade, oferecendo uma solução que pode ser adaptada para vários projetos.

FUNCIONALIDADES

O sistema será capaz de realizar:

ID	Funcionalidade
RF-01	Cadastro e login de usuários
RF-02	Avaliações de estabelecimentos
RF-03	Simulações de pedidos / entregas
RF-04	Visualização de dados por parte dos estabelecimentos
RF-05	Cadastro, edição e exclusão de produtos
RF-06	Cadastro e login de responsáveis por restaurantes
RF-07	Cadastro e manutenção de dados do restaurante
RF-08	Controle de estoque e preços dos produtos
RF-09	Alteração de preços, imagens, nomes dos produtos, descrições e categorias

Nota de consistência — RF-02: esta tabela descreve avaliações de estabelecimentos. Nas seções de modelo de dados e regras de avaliação, a entidade `Review` está vinculada a `OrderItem` e representa avaliação de produto. O escopo definitivo de RF-02 permanece pendente de validação; as duas descrições são preservadas até essa decisão.

O sistema não terá:

- Integração com gateway de pagamento
- Autenticação de dois fatores
- Integração operacional com entregadores, rastreamento por GPS ou despacho real.
- Promessa de entrega em prazo real.
- Emissão de documentos fiscais.
- Programa de fidelidade, cupons ou recomendações automatizadas.

Permissões

- Usuários
    - Cadastro e login
    - Avaliação de estabelecimentos
    - Simulações de pedidos / entregas
    - Alteração de seus próprios dados pessoais
- Restaurante
    - Cadastro e login
    - Cadastro e exclusão de produtos
    - Alteração de preços, imagens, nomes dos produtos, descrições e categorias
    - Visualização de dados como quantidade de produtos vendidos, avaliações recebidas

Restrições e premissas

- Pedidos e entregas serão apenas demonstrativos.
- Não terá integração com pagamento real.
- O aplicativo será desenvolvido apenas para dispositivos móveis, e o sistema web será para visualização de dados, indicadores e gestão de estoque para os restaurantes.
- O código será publicado como código aberto.

Regras de negócio

ID	Regra de negócio
RN-01	Cada usuário só poderá realizar pedidos em um restaurante por vez
RN-02	Um restaurante pode ter apenas um produto com o mesmo nome

ID	Regra de negócio
RN-03	Um responsável só pode gerenciar produtos associados a um restaurante para o qual tenha permissão
RN-04	Cada conta só pode atualizar os próprios dados pessoais
RN-05	Uma avaliação precisa estar associada a um cliente identificado
RN-06	Não deve ser possível adicionar um produto sem estoque ao carrinho
RN-07	Um produto excluído não deve aparecer como opção nova em pedidos
RN-08	Um usuário poderá adicionar produtos ao carrinho apenas se o restaurante estiver aberto e se o usuário estiver logado no sistema
RN-09	Ao remover um produto do carrinho, o sistema deve atualizar o valor total do pedido

Nota de consistência — RN-02: a regra estabelece um único produto com o mesmo nome por restaurante, mas não define se a comparação ignora diferenças entre maiúsculas/minúsculas, espaços, acentos ou outras formas de normalização. A documentação também utiliza `ProductBase` compartilhado e `RestaurantProduct` por restaurante; a entidade e o nível em que a unicidade será aplicada precisam ser confirmados. Nenhum critério de normalização está definido.

Estados do pedido

- Fluxo:
  - CRIADO → CONFIRMADO_RESTAURANTE → EM_PREPARO → SAIU_PARA_ENTREGA → CONCLUÍDO
  - O status "CANCELADO" poderá ser aplicado em qualquer etapa do fluxo, desde que o pedido ainda não tenha sido concluído.

Nota de consistência — cancelamento: a regra geral acima permite cancelamento em qualquer etapa anterior à conclusão. A seção detalhada do fluxo de pedidos estabelece condições mais específicas por papel e estado: o cliente pode cancelar em `CRIADO`, `CONFIRMADO_RESTAURANTE` e `EM_PREPARO`; o responsável pode cancelar apenas em `CONFIRMADO_RESTAURANTE`. As definições entram em conflito e nenhuma delas será escolhida como definitiva nesta revisão. Consulte a seção de pendências ao final.

Requisitos Não Funcionais

ID	Requisito Não Funcional
RNF-01	As telas mais utilizadas devem responder em tempo aceitável sob uma carga de 1000 usuários simultâneos
RNF-02	Falhas de comunicação exibem mensagem compreensível e não simulam sucesso

Esclarecimento de testabilidade: o RNF-01 estabelece a carga de 1.000 usuários simultâneos, mas não define quais telas/operações compõem a medição, o limite de tempo de resposta, os percentis, a taxa máxima de erro, a duração do teste ou o ambiente e conjunto de dados. O RNF-02 determina a apresentação de mensagem compreensível e a ausência de indicação falsa de sucesso; não detalha todas as classes de falha, a política de repetição ou os critérios de encerramento da tentativa. Essas informações permanecem pendentes, sem atribuição de valores nesta revisão.

Permissões

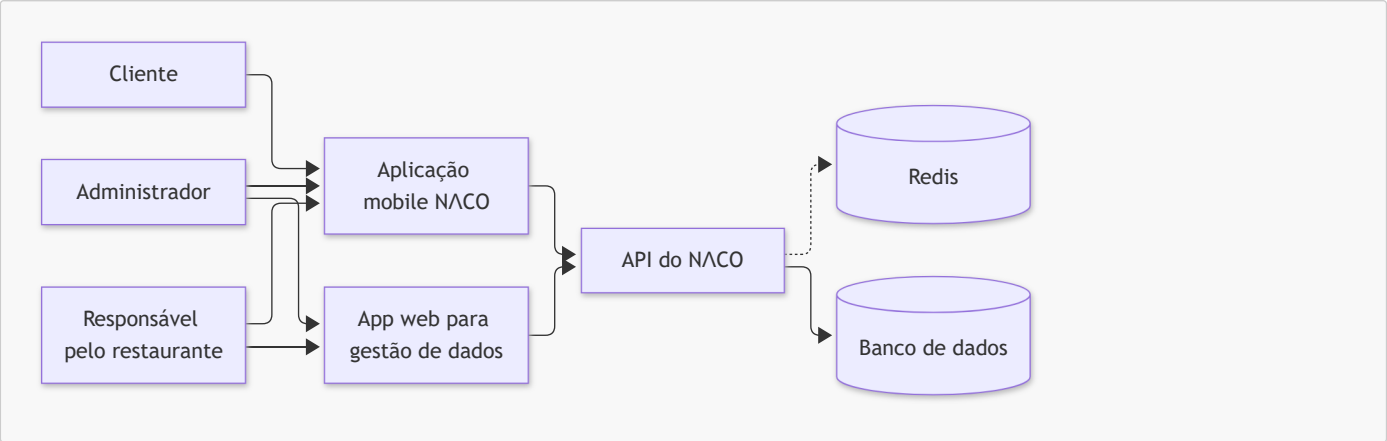
Ação	Visitante	Cliente autenticado	Responsável do restaurante	Administrador da plataforma
Consultar restaurantes e produtos públicos	Sim	Sim	Sim	Sim
Criar conta / iniciar sessão	Sim	Sim	Sim	Sim
Atualizar dados pessoais próprios	Não	Sim	Sim	Sim
Criar avaliação	Não	Sim	Não	Não
Criar pedido simulado	Não	Sim	Não	Apenas em ambiente de teste
Criar ou editar produtos	Não	Não	Apenas no restaurante autorizado	Sim, mediante autorização prévia
Excluir/desativar produtos	Não	Não	Apenas no restaurante autorizado	Sim, mediante autorização prévia
Consultar indicadores do restaurante	Não	Não	Apenas no restaurante autorizado	Sim, mediante autorização prévia
Moderar contas ou conteúdo	Não	Não	Não	Sim

Nota sobre o papel `ADMIN`: a matriz registra as permissões acima, incluindo operações condicionadas a autorização prévia e criação de pedidos apenas em ambiente de teste. O modelo de dados também informa que as permissões administrativas detalhadas ainda precisam ser definidas. Portanto, a matriz deve ser tratada como o conjunto de ações já mencionadas, mas os limites, o procedimento de autorização prévia e o alcance de cada ação continuam pendentes de validação.

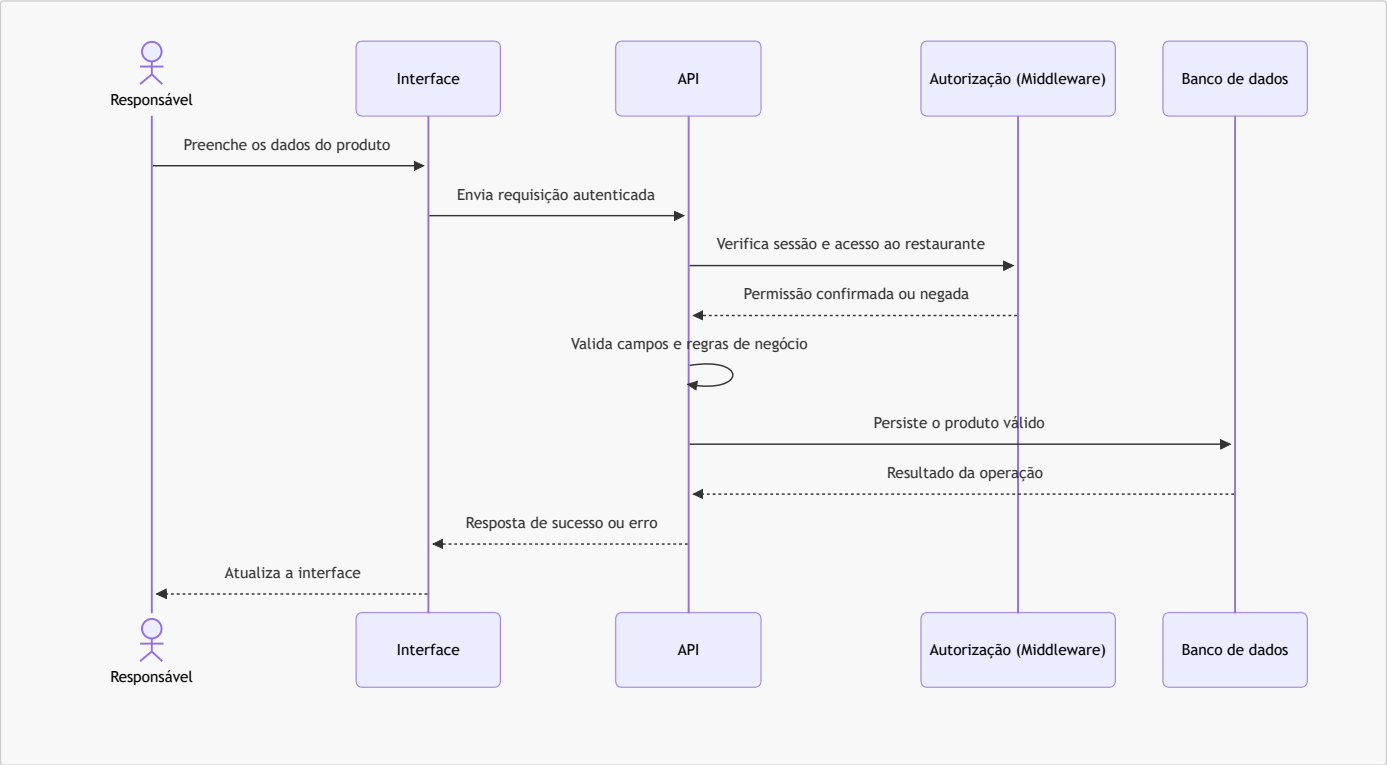
Arquitetura proposta

- Com base em pesquisas, decidimos começar o desenvolvimento iniciando uma arquitetura monolítica e escalar para uma arquitetura de microsserviços conforme surgir a necessidade. Outro motivo seria evitar uma complexidade desnecessária.

Comunicação entre os serviços



Fluxo de dados de exemplo — criar produto



Tecnologias e ferramentas

As escolhas abaixo servem como exemplo de preenchimento, não como uma recomendação definitiva. Compare as opções e registre a razão da decisão antes de instalar serviços ou bibliotecas por hábito.

Área	Opção	Critério para decidir
Interface mobile	React Native com TypeScript	Familiaridade e facilidade de manutenção
Interface web	React com TypeScript	Familiaridade
Backend	Node.js + Express.js	Organização da API, simplicidade e manutenção
API	REST sobre HTTP (com possibilidade de mudança para Server-Sent Events ou websockets)	Simplicidade dos fluxos e facilidade para documentar/testar
Banco de dados	PostgreSQL + Prisma	Familiaridade, maior robustez e integridade de dados (Também cheguei a considerar MySQL, mas nesses pontos o PostgreSQL é melhor.)
Cache	Redis	Simplicidade, performance e redução de consultas ao banco de dados
Acesso a dados	Prisma	Simplicidade, tipagem forte e familiaridade
Autenticação	Autenticação JWT	Robustez e segurança
Imagens	Cloudinary	Facilidade, otimização
Testes de API	Postman	Familiaridade
Testes de Carga	Grafana K6	Estabilidade, escalabilidade e performance

Área	Opção	Critério para decidir
Testes de interface	Appium	Necessidade de validar fluxos reais de navegação
Versionamento	Git/Github	Histórico, revisão e familiaridade
CI	GitHub Actions	Reduzir regressões sem criar manutenção excessiva

Mapa inicial de telas

Tela	Acesso	Conteúdo principal
Explorar restaurantes	Visitante/cliente	Lista de estabelecimentos e filtros iniciais
Detalhes do restaurante	Visitante/cliente	Perfil, categorias, produtos e avaliações
Detalhe do produto	Visitante/cliente	Descrição, preço e informações relevantes
Cadastro e login	Visitante	Formulários e mensagens de validação
Perfil	Cliente	Dados pessoais e edição permitida
Carrinho/resumo	Cliente	Itens, quantidades, totais e aviso de simulação
Pedido simulado	Cliente	Confirmação e estado demonstrativo
Visão geral	Restaurante	Dados sobre vendas e estoque
Produtos	Restaurante	Listagem, cadastro, edição e exclusão/desativação de produtos
Avaliações	Restaurante	Notas e comentários recebidos

Organização do frontend

- A navegação será implementada usando a biblioteca React Navigation, além da implementação de paginação para a listagem de produtos e restaurantes. Entre as páginas, dependendo do contexto, serão compartilhados dados como o token e o ID do usuário, o ID do restaurante e do produto.
- A comunicação com a API será feita usando a biblioteca Axios, que retornará códigos de status HTTP conforme cada caso. Em caso de falha, o usuário será redirecionado para uma tela de erro com as opções de voltar ou tentar novamente (com rate limiting). Com o tempo, a ideia é implementar mensageria em tempo real, tanto para casos de 2 pedidos simultâneos quanto para atualizações de status de pedidos. O cache será responsável por armazenar dados de restaurantes e produtos, atualizando sempre que houver alterações de alguma informação.
- Os formulários validarão os campos obrigatórios e os dados informados tanto via frontend quanto via backend, retornando mensagens de erro ou sucesso. Será avaliado o uso de bibliotecas de validação de formulários, como Formik ou React Hook Form.
- No dispositivo, serão armazenados dados como o token de autenticação, permitindo que o usuário permaneça logado após a saída do aplicativo. Os dados ficarão persistidos no armazenamento local do dispositivo por um período indeterminado.

Nota de segurança: a persistência por período indeterminado é uma definição presente na documentação atual. A duração efetiva do JWT, renovação, revogação, encerramento de sessão e requisitos de armazenamento seguro não estão suficientemente especificados. A presente revisão não altera o comportamento registrado; esses detalhes permanecem pendentes de validação.

Backend e contrato da API

- O backend será responsável por autenticar usuários, aplicar permissões, validar as regras de negócio, calcular dados confiáveis, persistir as operações e responder com erros consistentes.

Rotas iniciais

Método	Rota ilustrativa	Objetivo	Acesso esperado
POST	`/api/auth/register`	Criar conta	Público, com rate limiting e proteção contra SQL injection
POST	`/api/auth/login`	Iniciar sessão	Público, com rate limiting e proteção contra SQL Injection
POST	`/api/auth/logout`	Encerrar sessão	Autenticado
GET	`/api/restaurants`	Listar estabelecimentos	Público
GET	`/api/restaurants/:id`	Consultar um estabelecimento	Público
GET	`/api/restaurants/:id/products`	Consultar catálogo	Público
POST	`/api/restaurants/:id/products`	Criar produto	Responsável pelo estabelecimento
PATCH	`/api/products/:id`	Atualizar produto	Responsável pelo estabelecimento
DELETE	`/api/products/:id`	Excluir ou desativar produto	Responsável pelo estabelecimento
POST	`/api/restaurants/:id/reviews`	Criar avaliação	Cliente autenticado
POST	`/api/orders/create`	Criar pedido	Cliente autenticado

Método	Rota ilustrativa	Objetivo	Acesso esperado
GET	`/api/orders/`	Consultar pedidos do cliente	Cliente autenticado
GET	`/api/restaurants/:restaurantId/orders`	Receber e aceitar pedido	Cliente proprietário
GET	`/api/restaurant/dashboard`	Consultar resumo do painel	Responsável autorizado

Nota de consistência — contratos iniciais: a rota `GET /api/restaurants/:restaurantId/orders` está descrita com a finalidade "Receber e aceitar pedido" e acesso "Cliente proprietário". A documentação não esclarece se o acesso esperado é do cliente proprietário do pedido ou do responsável pelo restaurante, nem como a operação de aceitar se relaciona ao método `GET`. A rota, o método e a descrição são preservados até validação. As rotas desta tabela são identificadas como iniciais/ilustrativas e devem ser comparadas com os contratos detalhados da seção de gerenciamento de restaurantes.

Formato de resposta

Erros:

- Exemplo conceitual de erro de validação para cadastro de produtos:

```
{
  "error": {
    "code": "422", // HTTP 422 Unprocessable Entity
    "message": "Revise os campos informados.",
    "fields": {
      "price": "O valor precisa ser maior que zero."
    }
  }
}
```

- Exemplo conceitual de erro de validação para login:

```
{
    "error": {
        "code": "401", // HTTP 401 Unauthorized
        "message": "Usuário ou senha inválidos."
    }
}
```

- Exemplo conceitual de acesso ao aplicativo sem conexão com internet:

```
{
    "error": {
        "code": "503", // HTTP 503 Service Unavailable
        "message": "Não foi possível estabelecer conexão com o servidor. Verifique sua conexão com a internet e tente
novamente."
    }
}
```

Sucesso:

- Exemplo conceitual de sucesso de login:

```
{
    "data": {
            "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEyMyIsIm5hbWUiOiJKb2huIERvZSIsImlhdCI6MTY4NzQyMDAwMCwiZXhwIjoxNjg3NDIzNjAwfQ.abc
123def456ghi789jkl012mno345pqr678stu901vwx234yz567890",
            "user": {
                "id": "123",
                "name": "John Doe",
                "email": "0Mv1o@example.com"
            }
        }
}
```

- Exemplo conceitual de sucesso de cadastro de produto:

```
{
    "data": {
            "id": "456",
```

```
            "name": "Pizza Margherita",
            "description": "Deliciosa pizza com molho de tomate, mussarela e manjericão.",
            "price": 19.99,
            "situation": "active",
            "restaurant": {
                "id": "789",
                "name": "Pizzaria do João"
            }
            }
}
```

Nota sobre os formatos de erro: os exemplos conceituais usam `422` para erro de validação, `401` para credenciais inválidas e `503` para indisponibilidade de comunicação. O contrato detalhado de gerenciamento de restaurantes também sugere `400` para dados inválidos e `404` para restaurante inexistente ou sem acesso. Além disso, o campo `error.code` aparece ora como código HTTP numérico em texto, ora como código semântico (por exemplo, `DUPLICATE_CNPJ`). A padronização desses formatos e a distinção entre HTTP status e código interno permanecem pendentes; os exemplos existentes não foram substituídos.

- Exemplo conceitual de sucesso de pedido:

```
{
    "data": {
            "id": "789",
            "status": "CRIADO",
            "total": 39.98,
            "items": [
                {
                    "productId": "456",
                    "name": "Pizza Margherita",
                    "quantity": 2,
                    "price": 19.99
                }
            ],
            "restaurant": {
                "id": "789",
                "name": "Pizzaria do João"
            }
        }
}
```

Modelo de Dados e Regras de Negócio — NΛCO

1. Entidades principais

1.1. Conta (`User`)

Representa uma conta de acesso ao sistema.

Principais atributos:

- `id`
- `name`
- `email` — único globalmente
- `password_hash`
- `status`
- `created_at`
- `updated_at`

Regras já definidas:

- Cada pessoa possui uma única conta na plataforma.
- O e-mail é único globalmente, independentemente dos papéis associados à conta.
- A senha pertence à conta, e não a cada papel individualmente.
- Os papéis da conta são registrados separadamente na entidade `UserRole`.
- Uma mesma conta pode acumular os papéis `CLIENTE` e `RESPONSAVEL`.
- Um responsável pode gerenciar vários restaurantes.
- Uma conta com o papel `RESPONSAVEL` pode existir sem nenhum restaurante cadastrado.

A validação de e-mail considera apenas a conta, sem permitir cadastros duplicados para papéis diferentes.

1.2. Papéis do usuário (`UserRole`)

Representa os papéis associados a uma conta de usuário.

Principais atributos:

- `user_id` — FK para `User`
- `role` — `CLIENTE`, `RESPONSAVEL` ou `ADMIN`

Chave primária:

- Chave composta por `user_id` e `role`.

Regras já definidas:

- Os papéis são associados à conta por meio de `UserRole`.
- Uma pessoa não precisa criar outra conta para exercer outro papel na plataforma.
- Uma conta pode possuir vários papéis.
- Não pode existir o mesmo papel duplicado para uma conta.
- As permissões de acesso devem considerar os papéis associados à conta.
- Para realizar pedidos, a conta deve possuir o papel `CLIENTE`.
- Para gerenciar restaurantes, a conta deve possuir o papel `RESPONSAVEL`.
- O papel `ADMIN` identifica contas com função administrativa na plataforma; suas permissões detalhadas ainda precisam ser definidas.

1.3. Restaurante (`Restaurant`)

Representa o estabelecimento cadastrado na plataforma.

Principais atributos:

- `id`
- `responsible_user_id` — FK para `User`
- `public_name`
- `legal_name` — opcional
- `cnpj` — único
- `description` — opcional
- `phone`
- `cep`
- `street`
- `number`
- `neighborhood`
- `city`
- `state`
- `address_complement` — opcional
- `cuisine_category`
- `opening_hours` — estrutura a definir
- `logo` — opcional
- `cover_image` — opcional
- `publication_status` — `PUBLISHED` ou `UNPUBLISHED`
- `created_at`
- `updated_at`

Regras já definidas:

- Cada restaurante possui exatamente um responsável.
- O responsável deve possuir o papel `RESPONSAVEL`.
- Um responsável pode possuir vários restaurantes.
- O cadastro do restaurante não depende de aprovação administrativa.
- O responsável decide quando publicar ou retirar o restaurante da plataforma.
- Todo restaurante recém-cadastrado começa com `publication_status = UNPUBLISHED`.
- A publicação é diferente do funcionamento: um restaurante pode estar publicado, mas fechado naquele momento.
- O estado de funcionamento (`operating_status`) será calculado a partir dos horários cadastrados, não sendo necessariamente um campo persistido.
- Um restaurante publicado e fechado pode continuar aparecendo nas consultas públicas, mas não pode receber pedidos.
- Não pode retirar o restaurante da publicação enquanto houver pedidos ativos.
- Para essa regra, são considerados ativos os pedidos em `CRIADO`, `CONFIRMADO_RESTAURANTE`, `EM_PREPARO` ou `SAIU_PARA_ENTREGA`.
- Pedidos concluídos, recusados ou cancelados não impedem a despublicação.
- Mesmo despublicado, o responsável continua acessando o painel e gerenciando seus dados.

1.4. Produto-base (`ProductBase`)

Representa um tipo de produto compartilhável entre restaurantes.

Por exemplo, diferentes restaurantes podem oferecer uma pizza de calabresa. Não precisamos cadastrar o conceito do produto novamente para cada estabelecimento.

Principais atributos:

- `id`
- `name`
- `description`
- `created_at`
- `updated_at`

1.5. Produto ofertado (`RestaurantProduct`)

Representa a oferta daquele produto em um restaurante específico.

Essa separação é importante porque cada restaurante pode ter seu próprio preço, estoque e imagem.

Principais atributos:

- `id`
- `restaurant_id` — FK para `Restaurant`
- `product_base_id` — FK para `ProductBase`
- `category_id` — FK para `Category`
- `price`
- `stock_quantity`
- `image` — opcional
- `status`
- `created_at`
- `updated_at`

Exemplo:

Produto	Restaurante	Preço	Estoque
Pizza de calabresa	Pizzaria A	R$ 45,00	10
Pizza de calabresa	Pizzaria B	R$ 52,00	5
Pizza de calabresa	Pizzaria C	R$ 39,00	0

O produto-base é compartilhado, mas cada restaurante controla sua própria oferta.

Regras já definidas:

- Cada restaurante controla seu próprio preço e estoque.
- Produtos sem estoque continuam visíveis no catálogo, mas ficam indisponíveis para compra.
- Cada produto ofertado pertence a um restaurante e a uma categoria desse mesmo restaurante.
- O preço deve ser maior que zero.
- O estoque não pode ser negativo.
- Alterações nos produtos não podem comprometer o histórico dos pedidos.
- A exclusão de um produto não poderá apagar ou invalidar os dados históricos dos pedidos que o utilizaram.
- Produtos que já foram utilizados em pedidos deverão ser desativados ou ter sua exclusão física restringida, preservando as referências históricas.

1.6. Categoria (`Category`)

Representa as categorias utilizadas para organizar os produtos de um restaurante.

Principais atributos:

- `id`
- `restaurant_id` — FK para `Restaurant`
- `name`
- `description` — opcional
- `display_order`
- `status`

Regras já definidas:

- Cada restaurante cria e administra suas próprias categorias.
- As categorias são livres e personalizadas por restaurante.
- Um produto ofertado só pode utilizar uma categoria pertencente ao mesmo restaurante.

Exemplo:

- Pizzaria A: Pizzas, Bebidas e Sobremesas.
- Restaurante B: Hambúrgueres, Porções e Combos.

Importante: a categoria do restaurante, como pizzaria ou hamburgueria, é diferente da categoria de um produto, como pizzas ou bebidas.

1.7. Pedido (`Order`)

Representa um pedido demonstrativo realizado por um cliente.

Principais atributos:

- `id`
- `customer_id` — FK para `User`
- `restaurant_id` — FK para `Restaurant`
- `status` — `CRIADO`, `CONFIRMADO_RESTAURANTE`, `EM_PREPARO`, `SAIU_PARA_ENTREGA`, `CONCLUIDO`, `RECUSADO` ou `CANCELADO`

- `total`
- `created_at`
- `updated_at`

Regras já definidas:

- Um pedido pertence a um único restaurante.
- O cliente precisa estar autenticado e possuir o papel `CLIENTE`.
- O restaurante precisa estar publicado e dentro do horário de funcionamento para receber novos pedidos.
- Após a criação, o pedido aparece no painel do restaurante.
- O responsável autorizado pode aceitar ou recusar o pedido.
- O pedido recusado permanece registrado no histórico do cliente.
- O pedido deve possuir pelo menos um item.
- O total do pedido deve corresponder à soma dos subtotais de seus itens.
- O preço e a disponibilidade dos produtos devem ser validados pelo backend no momento da criação do pedido.
- O status inicial de todo pedido será `CRIADO`.
- O estoque só é reduzido quando o restaurante confirma o pedido.

1.8. Itens do pedido (`OrderItem`)

Armazena os produtos e quantidades incluídos em cada pedido.

Principais atributos:

- `id`
- `order_id` — FK para `Order`
- `restaurant_product_id` — FK para `RestaurantProduct`
- `product_name` — snapshot do nome no momento do pedido
- `unit_price` — snapshot do preço no momento do pedido
- `quantity`
- `subtotal`

Um detalhe importante: o nome e o preço devem ser registrados como uma fotografia dos valores no momento do pedido.

Assim, se o restaurante alterar o preço de uma pizza amanhã, o histórico de um pedido realizado hoje continuará exibindo o preço original.

Regras já definidas:

- Cada item deve pertencer ao mesmo restaurante associado ao pedido.
- A quantidade mínima por item é 1.
- A quantidade máxima por produto em cada pedido é 20 unidades.
- Não será permitido adicionar produtos sem estoque ao carrinho.
- O mesmo `restaurant_product_id` não poderá aparecer em mais de uma linha do mesmo pedido.
- A combinação de `order_id` e `restaurant_product_id` deverá ser única.
- O `subtotal` deverá corresponder ao preço unitário registrado multiplicado pela quantidade.
- O nome e o preço deverão ser preservados como snapshots, independentemente de alterações posteriores no catálogo.

1.9. Avaliação de produto (`Review`)

Armazena as avaliações feitas pelos clientes sobre produtos que adquiriram.

Principais atributos:

- `id`
- `order_item_id` — FK para `OrderItem`
- `rating`
- `comment`
- `created_at`
- `updated_at`

Regras já definidas:

- Somente clientes autenticados podem realizar avaliações.
- A avaliação deve estar vinculada a um item específico de um pedido realizado pelo cliente.
- O cliente só pode avaliar produtos que realmente estejam presentes em seus pedidos.
- A avaliação só pode ser realizada depois que o pedido atingir o status `CONCLUIDO`.
- A validação deve confirmar que o item pertence a um pedido do cliente autenticado.
- A nota deve estar entre 1 e 5.
- O cliente responsável pela avaliação será identificado pelo relacionamento entre `Review`, `OrderItem` e `Order`, sem necessidade de armazenar `customer_id` novamente em `Review`.

A avaliação é vinculada ao item do pedido, e não diretamente ao restaurante. A possibilidade de uma avaliação por item será implementada como restrição única em `order_item_id` caso essa regra seja mantida; essa decisão permanece pendente.

Nota de consistência — escopo de avaliações: esta entidade documenta avaliações de produtos. O requisito RF-02 e a descrição inicial de permissões também mencionam avaliações de estabelecimentos. Não há definição suficiente para concluir se RF-02 abrange apenas estabelecimentos, apenas produtos ou ambos. A relação

entre o endpoint `POST /api/restaurants/:id/reviews` e a entidade `Review` vinculada a `OrderItem` também depende dessa decisão.

1.10. Histórico de eventos do pedido (`OrderEvent`)

Registra as alterações de estado de cada pedido.

Principais atributos:

- `id`
- `order_id` — FK para `Order`
- `previous_status`
- `new_status`
- `changed_by` — FK para `User`
- `created_at`

Isso permite consultar quem alterou o pedido, quando ocorreu a alteração e qual era o estado anterior.

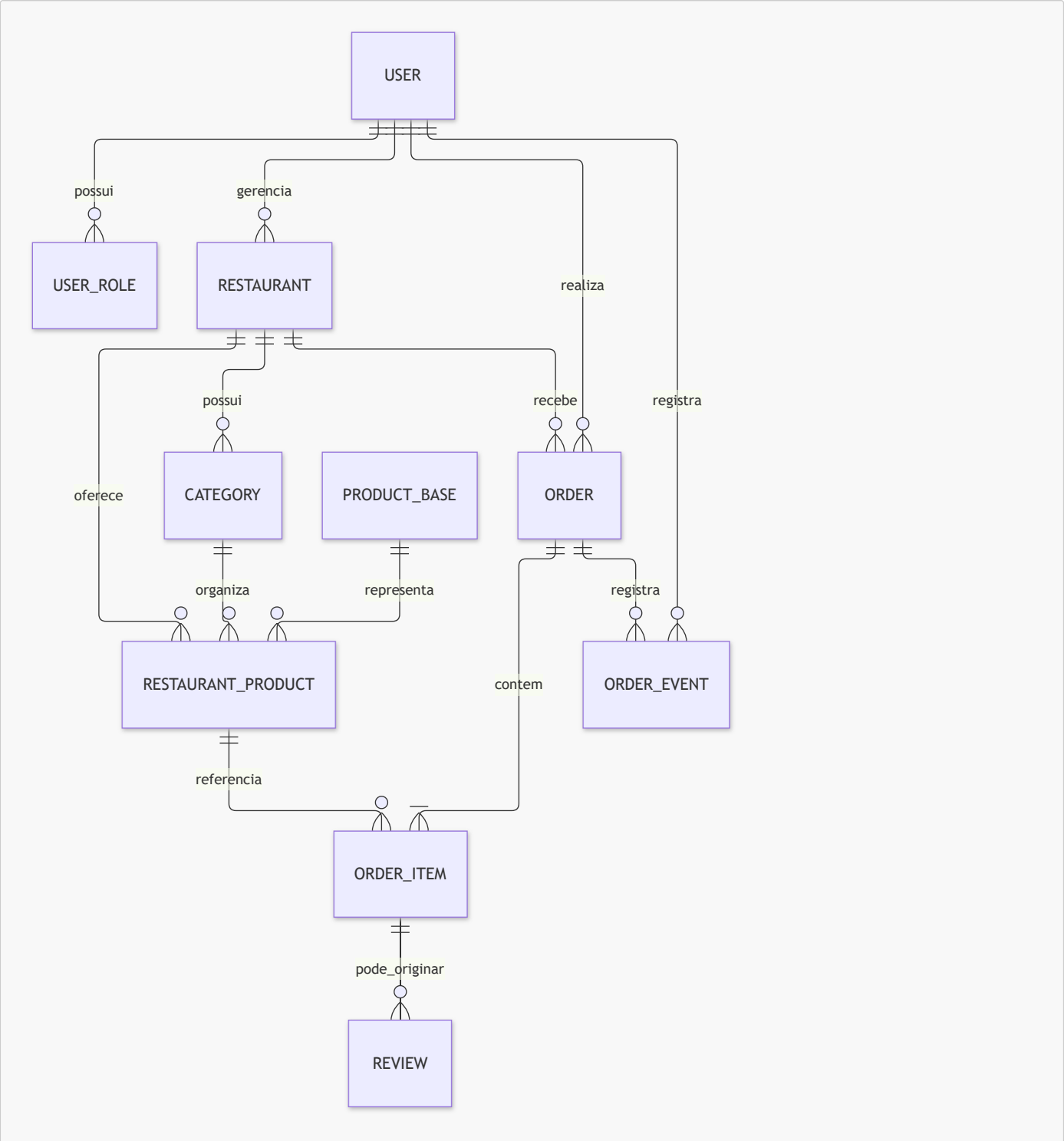
É especialmente útil para auditoria e para reconstruir o histórico de um pedido.

Regra definida: cada alteração de status deverá gerar um evento com o estado anterior, o novo estado, o usuário responsável e a data do registro.

---

2. Relacionamentos entre as entidades

A estrutura conceitual fica assim:



Esse é um modelo conceitual: os relacionamentos e as entidades estão representados, mas ainda não estamos fixando definitivamente todos os tipos de dados e mecanismos de implementação. As restrições de integridade definidas neste documento deverão ser aplicadas no banco de dados e/ou no backend, conforme indicado na seção 6.

---

3. Regras do fluxo que impactam a modelagem

Também já definimos algumas regras importantes para o funcionamento do banco:

Estados do pedido:

```
CRIADO
   |
   | Restaurante aceita
   v
CONFIRMADO_RESTAURANTE
   |
   v
EM_PREPARO
   |
```

```
   v
SAIU_PARA_ENTREGA
   |
   v
CONCLUIDO
```

Existem dois caminhos alternativos:

- `RECUSADO`: quando o restaurante não aceita o pedido.
- `CANCELADO`: quando o pedido é cancelado dentro das condições permitidas.

Divergência registrada: a regra geral inicial diz que `CANCELADO` pode ser aplicado em qualquer etapa anterior à conclusão. A definição detalhada abaixo restringe a operação por papel e estado. O quadro específico é mantido como está documentado, mas não é declarado como substituição da regra geral nesta revisão. A decisão final continua pendente.

Cancelamento:

- O cliente poderá cancelar enquanto o pedido estiver em `CRIADO`, `CONFIRMADO_RESTAURANTE` ou `EM_PREPARO`.
- O responsável poderá cancelar um pedido em `CONFIRMADO_RESTAURANTE`, mas não poderá cancelá-lo depois do início da preparação (`EM_PREPARO`).
- Não será permitido cancelar pedidos em `SAIU_PARA_ENTREGA` ou `CONCLUIDO`.

Estoque:

- Produtos sem estoque continuam visíveis, mas indisponíveis para compra.
- O estoque não é reduzido quando o cliente cria o pedido.
- A redução ocorre quando o restaurante confirma o pedido.
- Se um pedido for cancelado após a confirmação, o estoque deverá ser restaurado.

Restaurante:

- Não recebe novos pedidos fora do horário de funcionamento.
- A publicação e o funcionamento são controles independentes.
- A despublicação não apaga pedidos nem altera seus estados.
- A despublicação é bloqueada enquanto houver pedidos ativos.

---

4. Fluxo de pedidos

Estados do pedido

O pedido seguirá o seguinte fluxo:

`CRIADO` → `CONFIRMADO_RESTAURANTE` → `EM_PREPARO` → `SAIU_PARA_ENTREGA` → `CONCLUIDO`

Também existem os estados alternativos:

- `RECUSADO`: quando o restaurante rejeita um pedido.
- `CANCELADO`: quando o cliente ou, nas condições permitidas, o responsável cancela um pedido.

Regras de criação e confirmação

- O cliente só poderá criar pedidos enquanto o restaurante estiver publicado e dentro do horário de funcionamento.
- Cada pedido poderá conter produtos de apenas um restaurante.
- Ao criar o pedido, seu status inicial será `CRIADO`.
- O pedido ficará disponível no painel de gestão do restaurante.
- O restaurante poderá aceitar ou recusar o pedido.
- Se recusado, o pedido permanecerá no histórico do cliente com o status `RECUSADO`.
- O backend deverá validar os preços e a disponibilidade do estoque antes de registrar o pedido, utilizando os valores oficiais do catálogo.
- O estoque será reduzido somente quando o restaurante aceitar o pedido e seu status mudar para `CONFIRMADO_RESTAURANTE`.
- Antes de confirmar, o backend deverá verificar se ainda existe estoque suficiente para todos os itens.
- A confirmação do pedido e a redução do estoque deverão ocorrer de forma atômica, evitando inconsistências em operações simultâneas.

Transições de status

1. `CRIADO` → `CONFIRMADO_RESTAURANTE`

Quando o restaurante aceitar o pedido:

- O backend verifica se o pedido está no status `CRIADO`.
- Verifica se o responsável autenticado possui o papel `RESPONSAVEL` e autorização para aquele restaurante.
- Verifica se existe estoque suficiente para os produtos solicitados.
- Atualiza o status para `CONFIRMADO_RESTAURANTE`.
- Reduz as quantidades correspondentes no estoque.
- Registra a alteração em `OrderEvent`.

2. `CRIADO` → `RECUSADO`

Quando o restaurante recusar o pedido:

- O backend verifica se o pedido está no status `CRIADO`.
- Verifica se o responsável autenticado possui o papel `RESPONSAVEL` e autorização para aquele restaurante.
- Atualiza o status para `RECUSADO`.
- O estoque não é alterado, pois ainda não havia sido reduzido.
- Registra a alteração em `OrderEvent`.

3. `CONFIRMADO_RESTAURANTE` → `EM_PREPARO`

Quando o restaurante iniciar a preparação:

- Somente o responsável autorizado pelo restaurante poderá realizar a alteração.
- O backend verifica se o pedido está no status `CONFIRMADO_RESTAURANTE`.
- Atualiza o status para `EM_PREPARO`.
- Registra a data e o horário da alteração em `OrderEvent`.
- Não ocorre nenhuma alteração no estoque, pois ele já foi reduzido na confirmação.

4. `EM_PREPARO` → `SAIU_PARA_ENTREGA`

Quando o pedido deixar o restaurante:

- O backend verifica se o pedido está no status `EM_PREPARO`.
- Verifica se o pedido pertence ao restaurante responsável pela operação.
- Atualiza o status para `SAIU_PARA_ENTREGA`.
- Registra a data e o horário da saída em `OrderEvent`.
- A partir desse momento, o cliente e o responsável não poderão mais cancelar o pedido.

A entrega será apenas demonstrativa, sem integração com entregadores ou rastreamento por GPS.

5. `SAIU_PARA_ENTREGA` → `CONCLUIDO`

Quando a entrega simulada chegar ao destino:

- O backend verifica se o pedido está no status `SAIU_PARA_ENTREGA`.
- Atualiza o status para `CONCLUIDO`.
- Registra a data e o horário da conclusão em `OrderEvent`.
- Finaliza o fluxo do pedido.

Regras de cancelamento

Estado atual	Cliente pode cancelar?	Responsável pode cancelar?
`CRIADO`	Sim	Não; poderá recusar
`CONFIRMADO_RESTAURANTE`	Sim	Sim
`EM_PREPARO`	Sim	Não
`SAIU_PARA_ENTREGA`	Não	Não
`CONCLUIDO`	Não	Não
`RECUSADO`	Não	Não
`CANCELADO`	Não	Não

- O pedido cancelado permanecerá disponível no histórico do cliente.
- Se o cancelamento ocorrer após a confirmação do restaurante, o estoque deverá ser restaurado.
- A restauração do estoque e a alteração do status deverão ocorrer de forma atômica, evitando devoluções duplicadas.
- Toda alteração de status decorrente de cancelamento deverá ser registrada em `OrderEvent`.

Histórico e visualização dos pedidos

- O cliente visualizará o status atual do pedido.
- O sistema deverá registrar internamente as alterações de status e seus respectivos horários para fins de consistência e rastreabilidade.
- Cada evento deverá identificar o pedido, o status anterior, o novo status, o usuário responsável pela alteração e a data do registro.
- Não serão implementadas notificações nesta versão inicial.

---

5. Avaliações de produtos

Regras de avaliação

- Somente clientes autenticados poderão realizar avaliações.
- O cliente poderá avaliar exclusivamente produtos que estejam presentes em pedidos realizados por ele.
- Não será permitido avaliar um produto que não tenha sido solicitado, mesmo que pertença ao mesmo restaurante.
- As avaliações só poderão ser realizadas depois que o pedido atingir o status `CONCLUIDO`.

- A avaliação deverá estar vinculada ao produto e ao respectivo item do pedido (`OrderItem`), permitindo verificar se o cliente realmente adquiriu aquele produto.
- O backend deverá validar a propriedade do pedido, a existência do produto entre seus itens e o status `CONCLUIDO` antes de permitir a avaliação.
- A avaliação deverá ser associada ao `order_item_id`; o cliente será identificado por meio do pedido relacionado.
- A nota deverá estar entre 1 e 5.
- A restrição de uma avaliação por item do pedido ainda depende da decisão sobre avaliações repetidas.

Observação: as regras acima descrevem avaliações de produtos vinculadas a `OrderItem`; o escopo de RF-02, que menciona avaliações de estabelecimentos, permanece em aberto.

Exemplo

Um cliente realizou um pedido contendo:

- Pizza de calabresa.
- Refrigerante.

Nesse caso, poderá avaliar os dois produtos, mas não poderá avaliar uma sobremesa que não tenha solicitado, mesmo que ela pertença ao mesmo restaurante.

---

6. Restrições gerais de integridade

Além das relações entre as entidades, teremos as seguintes regras de integridade:

Regra	Onde será validada
E-mail único	Banco e backend
Papéis sem duplicidade	Banco
CNPJ único	Banco e backend
Preço maior que zero	Banco e backend
Estoque não negativo	Banco e backend
Quantidade entre 1 e 20	Banco e backend
Um item por produto em cada pedido	Banco e backend
Pedido com pelo menos um item	Backend e transação
Subtotal consistente com preço e quantidade	Backend e/ou banco
Avaliação entre 1 e 5	Banco e backend
Produto e categoria do mesmo restaurante	Banco e backend
Cliente proprietário do pedido	Backend
Responsável proprietário do restaurante	Backend
Transições válidas de status	Backend
Operações de estoque atômicas	Banco e backend

A combinação de `order_id` e `restaurant_product_id` em `OrderItem` deverá possuir uma restrição de unicidade, impedindo que o mesmo produto seja dividido em várias linhas do mesmo pedido para contornar o limite de 20 unidades.

A integridade entre `RestaurantProduct` e `Category` deverá garantir que ambos pertençam ao mesmo `restaurant_id`.

Resultado da revisão

Temos uma estrutura consolidada inicial com as entidades:

1. `User`
2. `UserRole`
3. `Restaurant`
4. `ProductBase`
5. `RestaurantProduct`
6. `Category`
7. `Order`
8. `OrderItem`
9. `Review`
10. `OrderEvent`

1. Cadastrar restaurante

`POST /api/restaurants`

Cria um restaurante vinculado ao responsável autenticado.

Request:

```
{
  "public_name": "Pizzaria do Gabriel",
  "legal_name": "Gabriel Hanel Pizzaria LTDA",
  "cnpj": "12345678000199",
  "description": "Pizzas artesanais",
  "phone": "54999999999",
  "cuisine_category": "PIZZARIA",
  "address": {
    "cep": "99950000",
    "street": "Rua das Flores",
    "number": "150",
    "neighborhood": "Centro",
    "city": "Tapejara",
    "state": "RS",
    "complement": "Sala 2"
  },
  "logo": null,
  "cover_image": null
}
```

Response — 201 Created:

```
{
  "id": "restaurant_123",
  "responsible_user_id": "user_456",
  "public_name": "Pizzaria do Gabriel",
  "legal_name": "Gabriel Hanel Pizzaria LTDA",
  "cnpj": "12345678000199",
  "description": "Pizzas artesanais",
  "phone": "54999999999",
  "cuisine_category": "PIZZARIA",
  "address": {
    "cep": "99950000",
    "street": "Rua das Flores",
    "number": "150",
    "neighborhood": "Centro",
    "city": "Tapejara",
    "state": "RS",
    "complement": "Sala 2"
  },
  "publication_status": "UNPUBLISHED",
  "created_at": "2026-10-31T18:00:00Z"
}
```

Validações do backend:

- Usuário autenticado.
- Usuário possui o papel `RESPONSAVEL`.
- CNPJ em formato válido e ainda não cadastrado.
- Telefone e CEP em formatos válidos.
- Campos obrigatórios preenchidos.
- O responsável é identificado pelo token de autenticação, nunca pelo corpo da requisição.
- O restaurante começa obrigatoriamente como `UNPUBLISHED`.

O horário de funcionamento será incorporado ao contrato depois que definirmos sua estrutura na etapa 4. A entidade `Restaurant` já inclui `opening_hours`, mas sua estrutura e os critérios para cálculo de `operating_status` ainda não estão definidos.

2. Listar restaurantes do responsável

`GET /api/me/restaurants`

Retorna todos os restaurantes vinculados ao responsável autenticado.

Response — 200 OK:

```
{
  "restaurants": [
    {
      "id": "restaurant_123",
      "public_name": "Pizzaria do Gabriel",
      "cuisine_category": "PIZZARIA",
      "publication_status": "UNPUBLISHED",
      "created_at": "2026-10-31T18:00:00Z"
```

```
    },
    {
      "id": "restaurant_456",
      "public_name": "Hamburgueria do Gabriel",
      "cuisine_category": "HAMBURGUERIA",
      "publication_status": "PUBLISHED",
      "created_at": "2026-10-30T15:00:00Z"
    }
  ],
  "total": 2
}
```

Regras:

- Retorna somente restaurantes do responsável autenticado.
- Não retorna restaurantes de outros responsáveis.
- Inclui restaurantes publicados e despublicados.
- Se não houver restaurantes, retorna uma lista vazia.

3. Consultar detalhes para gestão

`GET /api/restaurants/{id}/management`

Retorna os dados completos de um restaurante para seu gerenciamento.

Exemplo:

`GET /api/restaurants/restaurant_123/management`

Response — 200 OK:

```
{
  "id": "restaurant_123",
  "public_name": "Pizzaria do Gabriel",
  "legal_name": "Gabriel Hanel Pizzaria LTDA",
  "cnpj": "12345678000199",
  "description": "Pizzas artesanais",
  "phone": "54999999999",
  "cuisine_category": "PIZZARIA",
  "address": {
    "cep": "99950000",
    "street": "Rua das Flores",
    "number": "150",
    "neighborhood": "Centro",
    "city": "Tapejara",
    "state": "RS",
    "complement": "Sala 2"
  },
  "logo": null,
  "cover_image": null,
  "publication_status": "UNPUBLISHED",
  "created_at": "2026-10-31T18:00:00Z",
  "updated_at": "2026-10-31T18:00:00Z"
}
```

Regras:

- Somente o responsável daquele restaurante pode acessar.
- O restaurante pode ser consultado mesmo despublicado.
- Não deve ser confundido com o endpoint público de visualização do restaurante.

4. Editar dados do restaurante

`PATCH /api/restaurants/{id}`

Atualiza parcialmente os dados cadastrais.

Request:

```
{
  "public_name": "Pizzaria Gabriel",
  "description": "Pizzas artesanais e tradicionais",
  "phone": "54988888888",
  "address": {
    "cep": "99950000",
    "street": "Rua das Flores",
```

```
    "number": "200",
    "neighborhood": "Centro",
    "city": "Tapejara",
    "state": "RS",
    "complement": null
  }
}
```

Response — 200 OK:

```
{
  "id": "restaurant_123",
  "public_name": "Pizzaria Gabriel",
  "description": "Pizzas artesanais e tradicionais",
  "phone": "54988888888",
  "publication_status": "UNPUBLISHED",
  "updated_at": "2026-10-31T18:30:00Z"
}
```

Regras:

- Somente o responsável autorizado pode editar.
- Apenas os campos enviados são atualizados.
- Campos omitidos permanecem inalterados.
- Os dados devem passar pelas mesmas validações do cadastro.
- A edição não altera automaticamente o estado de publicação.

5. Publicar ou despublicar restaurante

`PATCH /api/restaurants/{id}/publication`

Endpoint exclusivo para controlar a publicação.

Request — publicar:

```
{
  "publication_status": "PUBLISHED"
}
```

Response — 200 OK:

```
{
  "id": "restaurant_123",
  "publication_status": "PUBLISHED",
  "updated_at": "2026-10-31T18:45:00Z"
}
```

Request — despublicar:

```
{
  "publication_status": "UNPUBLISHED"
}
```

Response — 200 OK:

```
{
  "id": "restaurant_123",
  "publication_status": "UNPUBLISHED",
  "updated_at": "2026-10-31T18:50:00Z"
}
```

Regras:

- Somente o responsável autorizado pode alterar a publicação.
- O restaurante começa como `UNPUBLISHED`.
- Não pode despublicar enquanto houver pedidos pendentes ou em andamento.
- Publicação não significa funcionamento: um restaurante publicado pode estar fechado.
- Despublicar não exclui produtos, pedidos ou histórico.

- O responsável continua tendo acesso ao gerenciamento.

Tratamento de erros da API

Sugiro padronizar as respostas de erro para todos os endpoints.

Exemplo:

```
{
  "error": {
    "code": "DUPLICATE_CNPJ",
    "message": "Já existe um restaurante cadastrado com este CNPJ."
  }
}
```

HTTP	Situação
400	Dados inválidos ou requisição incorreta
401	Usuário não autenticado
403	Usuário sem permissão
404	Restaurante não encontrado ou sem acesso
409	Conflito, como CNPJ duplicado ou tentativa de despublicação com pedidos em andamento
500	Erro interno do servidor

Nota de consistência: neste contrato detalhado, dados inválidos são associados a HTTP 400; nos exemplos conceituais anteriores, uma validação de produto utiliza HTTP 422. O contrato também sugere HTTP 404 para recurso inexistente ou sem acesso, enquanto a tabela de permissões e outros trechos distinguem ausência de autenticação (401) e falta de autorização (403). Os códigos acima são preservados como propostos, mas a política uniforme de respostas ainda precisa ser validada.

Resumo dos endpoints

Método	Endpoint	Finalidade
POST	`/api/restaurants`	Cadastrar restaurante
GET	`/api/me/restaurants`	Listar restaurantes do responsável
GET	`/api/restaurants/{id}/management`	Consultar detalhes
PATCH	`/api/restaurants/{id}`	Editar cadastro
PATCH	`/api/restaurants/{id}/publication`	Publicar ou despublicar

```
naco/
│
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
│
├── src/
│   │
│   ├── config/
│   │   ├── env.ts
│   │   ├── database.ts
│   │   └── app.ts
│   │
│   ├── modules/
│   │   │
│   │   ├── users/
│   │   │   ├── user.controller.ts
│   │   │   ├── user.service.ts
│   │   │   ├── user.repository.ts
│   │   │   ├── user.schema.ts
│   │   │   └── user.routes.ts
│   │   │
│   │   ├── restaurants/
│   │   │   ├── restaurant.controller.ts
│   │   │   ├── restaurant.service.ts
│   │   │   ├── restaurant.repository.ts
│   │   │   ├── restaurant.schema.ts
│   │   │   └── restaurant.routes.ts
│   │   │
│   │   ├── products/
```

```
│   │   │   ├── product-base.controller.ts
│   │   │   ├── product-base.service.ts
│   │   │   ├── product-base.repository.ts
│   │   │   ├── product-base.schema.ts
│   │   │   ├── restaurant-product.controller.ts
│   │   │   ├── restaurant-product.service.ts
│   │   │   ├── restaurant-product.repository.ts
│   │   │   ├── restaurant-product.schema.ts
│   │   │   └── product.routes.ts
│   │   │
│   │   ├── categories/
│   │   │   ├── category.controller.ts
│   │   │   ├── category.service.ts
│   │   │   ├── category.repository.ts
│   │   │   ├── category.schema.ts
│   │   │   └── category.routes.ts
│   │   │
│   │   ├── orders/
│   │   │   ├── order.controller.ts
│   │   │   ├── order.service.ts
│   │   │   ├── order.repository.ts
│   │   │   ├── order.schema.ts
│   │   │   ├── order-item.service.ts
│   │   │   ├── order-event.service.ts
│   │   │   └── order.routes.ts
│   │   │
│   │   └── reviews/
│   │       ├── review.controller.ts
│   │       ├── review.service.ts
│   │       ├── review.repository.ts
│   │       ├── review.schema.ts
│   │       └── review.routes.ts
│   │
│   ├── middlewares/
│   │   ├── auth.middleware.ts
│   │   ├── role.middleware.ts
│   │   ├── validate.middleware.ts
│   │   ├── error.middleware.ts
│   │   └── not-found.middleware.ts
│   │
│   ├── lib/
│   │   ├── errors/
│   │   │   ├── app-error.ts
│   │   │   └── error-codes.ts
│   │   ├── http/
│   │   │   └── response.ts
│   │   └── utils/
│   │       ├── cnpj.ts
│   │       └── pagination.ts
│   │
│   ├── routes/
│   │   └── index.ts
│   │
│   └── server.ts
│
├── tests/
│   ├── unit/
│   │   ├── users/
│   │   ├── restaurants/
│   │   ├── products/
│   │   ├── categories/
│   │   ├── orders/
│   │   └── reviews/
│   │
│   ├── integration/
│   │   ├── users/
│   │   ├── restaurants/
│   │   ├── products/
│   │   ├── categories/
│   │   ├── orders/
│   │   └── reviews/
│   │
│   └── fixtures/
│
├── .env
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
└── README.md
```

Camada	Responsabilidade
`controller`	Receber requisições HTTP e devolver respostas.
`service`	Implementar regras de negócio e coordenar operações.
`repository`	Centralizar consultas e operações no banco de dados.
`schema`	Definir validações e formatos de entrada.
`routes`	Definir endpoints e associar middlewares aos controllers.

Critérios de aceitação derivados dos requisitos existentes

Esta seção organiza condições verificáveis a partir dos requisitos já registrados. Quando a documentação não define um dado, limite, resposta ou comportamento, o item é identificado como pendente em vez de receber uma regra presumida.

Requisito	Condição inicial e entrada	Resultado esperado documentado	Verificação negativa ou limite	Informação ainda necessária
RF-01 — Cadastro e login de usuários	Acesso às rotas `POST /api/auth/register` e `POST /api/auth/login`.	Cadastro cria uma conta; login bem-sucedido retorna token e dados do usuário, conforme exemplo.	Credenciais inválidas retornam exemplo de erro HTTP 401.	Campos e validações completas do cadastro, resposta de cadastro, regras de senha e política de sessão.
RF-02 — Avaliações de estabelecimentos	A documentação menciona avaliações de estabelecimentos e também avaliação de produtos vinculada a `OrderItem`.	O comportamento esperado não pode ser consolidado enquanto o escopo estiver divergente.	Não é possível definir um teste de aceitação único sem decidir o objeto da avaliação.	Confirmar se RF-02 inclui avaliações de estabelecimento, de produto ou ambas e relacionar o endpoint à entidade correspondente.
RF-03 — Simulações de pedidos/entregas	Cliente autenticado; pedido com produtos de um único restaurante; restaurante publicado e dentro do horário; produtos disponíveis.	O pedido é criado com status `CRIADO`; total e itens são apresentados; pedidos e entregas são demonstrativos.	Usuário não autenticado, restaurante não publicado/fechado, produto sem estoque ou pedido com mais de um restaurante não podem ser aceitos como pedido válido.	Regras finais de cancelamento; contrato completo de entrada/saída; comportamento do carrinho e tratamento uniforme dos erros.
RF-04 — Visualização de dados por estabelecimentos	Responsável autorizado consulta o painel.	A documentação cita quantidade de produtos vendidos e avaliações recebidas e a rota `GET /api/restaurant/dashboard`.	Usuário sem autorização não deve consultar os dados do restaurante.	Campos, filtros, período, paginação, definição dos indicadores e contrato detalhado da resposta.
RF-05 — Cadastro, edição e exclusão de produtos	Responsável autorizado opera produtos do restaurante ao qual está vinculado.	Produtos podem ser cadastrados, editados e excluídos/desativados conforme os endpoints iniciais.	Cliente não pode editar produto; produto com histórico não pode ter sua referência histórica invalidada.	Corpo completo de cadastro/edição, distinção entre exclusão física e desativação em todos os casos e contrato de resposta.
RF-06 — Cadastro e login de responsáveis	Conta autenticada com papel `RESPONSAVEL`; autenticação usa as rotas gerais de conta.	Uma conta pode acumular papéis e um responsável pode administrar vários restaurantes.	Conta sem o papel `RESPONSAVEL` não satisfaz o requisito de gerenciamento de restaurante.	Fluxo/endpoint para atribuição do papel `RESPONSAVEL` e critérios específicos de cadastro de responsável.
RF-07 — Cadastro e manutenção dos dados do restaurante	Responsável autenticado e autorizado utiliza os endpoints de gerenciamento.	Criar, listar, consultar, editar e publicar/despublicar conforme os contratos detalhados; novo restaurante inicia `UNPUBLISHED`.	Não autenticado, papel ausente, CNPJ duplicado, dados obrigatórios inválidos ou tentativa de despublicação com pedidos ativos devem ser tratados conforme as regras e respostas documentadas.	Estrutura de `opening_hours`; conciliação de HTTP 400/422 e 403/404; detalhes de atualização parcial de objetos aninhados.
RF-08 — Controle de estoque e preços	Produto ofertado com preço e estoque cadastrados; cliente consulta e tenta comprar.	Preço maior que zero; estoque não negativo; produto sem estoque permanece visível e indisponível; estoque reduz somente na confirmação.	Estoque insuficiente impede a confirmação; preço e estoque são revalidados pelo backend.	Precisão decimal, arredondamento, moeda e estratégia técnica de concorrência/isolamento.
RF-09 — Alteração de preços, imagens, nomes, descrições e categorias	Responsável autorizado atualiza os campos do produto.	Campos atualizados refletem no catálogo; snapshots de nome/preço já registrados em pedidos permanecem preservados.	Atualização não pode alterar o histórico de pedidos; não pode gerenciar produto de restaurante sem autorização.	Contrato completo de atualização e regra de normalização/duplicidade de nomes.

Critérios verificáveis adicionais das regras de negócio

Regra	Verificação que decorre do texto existente	Limitação ou pendência
RN-01	Um pedido não pode conter produtos de mais de um restaurante.	Não há contrato específico de carrinho documentado.
RN-02	Verificar duplicidade de nome dentro do restaurante.	Comparação de caixa, espaços, acentos, normalização e entidade em que a unicidade se aplica não foram definidos.
RN-03	Responsável só gerencia produtos de restaurante autorizado.	A matriz de recursos e a verificação por recurso devem ser detalhadas para todas as operações.
RN-04	Conta só atualiza os próprios dados pessoais.	Campos editáveis e contrato de atualização do perfil não estão detalhados.
RN-05	Avaliação vinculada a cliente identificado.	Há divergência entre avaliação de estabelecimento e avaliação de produto; ver pendência de escopo.
RN-06	Produto sem estoque não pode ser adicionado ao carrinho.	A documentação também determina que produto sem estoque permaneça visível e indisponível.
RN-07	Produto excluído não aparece como nova opção de pedido.	Exclusão física de produto já referenciado deve preservar o histórico; a estratégia definitiva por estado do produto não está detalhada além da restrição descrita no modelo.
RN-08	Adicionar produto ao carrinho depende de autenticação e restaurante aberto.	Critérios de horário, exceções e fuso/virada de dia não estão definidos; a criação do pedido também exige restaurante publicado.
RN-09	Remover item do carrinho recalcula o total.	O endpoint e a persistência do carrinho não estão descritos.

Cenários de teste documentados ou derivados diretamente

Os cenários abaixo verificam comportamentos já registrados. Os que dependem de decisões pendentes estão identificados como bloqueados para aprovação definitiva.

| Área | Cenário | Resultado/critério existente | Situação | |---|---|---|---|---| | Autenticação | Tentar acessar operação autenticada sem token/sessão. | Acesso negado; a documentação associa usuário não autenticado a HTTP 401 em contratos de gerenciamento. | Testável, observando a padronização de erros pendente. | | Autorização | Cliente tenta criar ou editar produto. | Operação não autorizada pela matriz de permissões. | Testável. | | Autorização | Responsável tenta consultar ou alterar restaurante de outro responsável. | A operação deve ser bloqueada pela verificação de propriedade/autorização. | Testável; código de erro para recurso sem acesso precisa ser padronizado. | | Produto | Tentar cadastrar preço igual ou inferior a zero. | Preço deve ser maior que zero. | Testável. | | Produto | Tentar persistir estoque negativo. | Estoque não pode ser negativo. | Testável. | | Produto | Cadastrar produto com nome duplicado no mesmo restaurante. | A regra RN-02 proíbe produto com o mesmo nome no restaurante. | Parcialmente bloqueado: normalização e entidade da restrição pendentes. | | Produto | Remover/desativar produto já presente em pedido. | Dados históricos e referências do pedido devem permanecer válidos. | Testável para preservação histórica; forma de exclusão em cada situação depende da estratégia documentada. | | Pedido | Cliente não autenticado tenta criar pedido. | Pedido exige cliente autenticado com papel `CLIENTE`. | Testável. | | Pedido | Tentar criar pedido com produtos de restaurantes diferentes. | Cada pedido pertence a um único restaurante. | Testável. | | Pedido | Tentar adicionar produto sem estoque ao carrinho. | A operação não deve permitir a inclusão; produto permanece visível como indisponível. | Testável. | | Pedido | Estoque torna-se insuficiente antes da confirmação. | Backend verifica estoque antes de confirmar; a confirmação e baixa devem ser atômicas. | Testável; resposta de erro específica não foi definida. | | Concorrência | Dois pedidos tentam consumir simultaneamente as últimas unidades. | A atomicidade e a validação de estoque devem impedir estoque inconsistente/negativo. | O resultado transacional deve ser testado; estratégia de isolamento, bloqueio e resposta da operação perdedora não estão especificadas. | | Cancelamento | Cliente tenta cancelar pedido em diferentes estados. | Há uma regra geral e uma matriz detalhada incompatíveis. | Bloqueado para aprovação final até decisão sobre a regra aplicável. | | Cancelamento | Responsável tenta cancelar pedido em diferentes estados. | A definição detalhada permite cancelamento apenas em `CONFIRMADO_RESTAURANTE`; a regra geral inicial é mais ampla. | Bloqueado para aprovação final até decisão sobre a regra aplicável. | | Avaliação | Cliente tenta avaliar item que não pertence a pedido seu. | Backend deve validar propriedade do pedido e item. | Testável para avaliação de produto; escopo geral de RF-02 ainda pendente. | | Avaliação | Tentar avaliar antes de pedido `CONCLUIDO`. | Avaliação de produto só ocorre após conclusão. | Testável para Review de produto. | | Avaliação | Tentar criar avaliação repetida para o mesmo item. | A restrição única foi indicada como condicional à decisão final. | Bloqueado até decidir se haverá uma avaliação por item. | | Restaurante | Criar restaurante sem autenticação ou sem papel `RESPONSAVEL`. | Criação exige autenticação e papel `RESPONSAVEL`. | Testável. | | Restaurante | Criar restaurante com CNPJ duplicado ou inválido. | Cadastro deve ser rejeitado; CNPJ é único e validado. | Testável; códigos HTTP/erro devem ser padronizados. | | Restaurante | Criar restaurante válido. | Deve associar o responsável autenticado e iniciar como `UNPUBLISHED`. | Testável, exceto campos/validações de `opening_hours` ainda não definidos. | | Restaurante | Responsável consulta restaurante despublicado. | Painel de gerenciamento continua acessível ao responsável. | Testável. | | Restaurante | Tentar despublicar com pedido ativo. | Operação deve ser bloqueada; estados ativos listados no modelo: `CRIADO`, `CONFIRMADO_RESTAURANTE`, `EM_PREPARO` e `SAIU_PARA_ENTREGA`. | Testável; confirmar consistência entre o conceito de pedido ativo e os estados/autorizações finais. | | Comunicação | Simular falha de conexão/indisponibilidade. | Interface exibe mensagem compreensível e não indica sucesso; exemplo documentado usa 503. | Testável quanto ao comportamento descrito; retry/rate limit e classes de falha pendentes. | | Desempenho | Executar teste com 1.000 usuários simultâneos. | Deve avaliar as telas mais utilizadas. | Teste pode ser executado, mas aprovação fica bloqueada até definir limite de resposta e demais métricas. |

Distinção de falhas para registro dos testes

A documentação já apresenta exemplos de erro de validação (`422`), credenciais inválidas (`401`), indisponibilidade (`503`), conflito (`409`), recurso não encontrado (`404`), falta de autorização (`403`) e erro interno (`500`). Esses exemplos podem ser usados para classificar os resultados observados, mas a correspondência definitiva entre cada cenário, status HTTP e formato de `error.code` ainda precisa ser padronizada.

Falhas de ambiente, infraestrutura ou comunicação devem ser registradas separadamente de falhas funcionais, sem que uma indisponibilidade seja reportada como aprovação da operação. A distinção é compatível com RNF-02; critérios detalhados de classificação ainda não foram definidos.

Pendências de especificação e validação

Tema	Localização	Informação ausente ou conflitante	Por que não deve ser presumida	Decisão necessária
Cancelamento	Estados do pedido; seção 4; critérios de cancelamento	Regra geral permite cancelar em qualquer etapa anterior à conclusão; regra detalhada diferencia cliente e responsável por estado.	As duas redações não podem ser simultaneamente aplicadas como uma única política.	Confirmar estados e papéis autorizados a cancelar e quais são os estados terminais.
Escopo das avaliações	RF-02; permissões; Review; endpoint de avaliações	RF-02 fala em avaliação de estabelecimento; modelo e regras descrevem avaliação de produto por `OrderItem`.	O objeto da avaliação altera o relacionamento de dados e a interpretação da rota existente.	Confirmar se serão avaliações de estabelecimentos, produtos ou ambos e como o endpoint existente se relaciona com o modelo.
Métricas do RNF-01	RNF-01	"Tempo aceitável" com 1.000 usuários simultâneos não possui limite mensurável.	Qualquer número acrescentado seria uma meta técnica nova.	Definir telas/operações, ambiente, massa de dados, duração, percentis/tempo máximo e taxa aceitável de erro.
Tratamento do RNF-02	RNF-02; organização do frontend	Mensagem compreensível e ausência de falso sucesso estão definidas; classes de falhas, tentativas, retry e rate limiting não estão detalhados.	Não há política suficiente para definir comportamento uniforme de repetição.	Definir quais falhas são cobertas, como a tentativa é encerrada e quais regras de retry/rate limiting se aplicam.
Nomes de produtos	RN-02; ProductBase; RestaurantProduct	Unicidade de nome não define normalização nem entidade/coluna na qual será aplicada.	Diferentes normalizações produzem resultados diferentes para nomes visualmente semelhantes.	Definir normalização e escopo da restrição de unicidade.
Horários de funcionamento	RN-08; Restaurant.opening_hours; RF-REST	Estrutura de horários está "a definir"; regras de exceção, virada de dia e fuso não constam.	Não há dados para calcular corretamente todos os casos de aberto/fechado.	Definir estrutura de horários e critérios de avaliação, inclusive exceções que venham a ser adotadas.
Valores monetários	Order.total; OrderItem.unit_price/subtotal; RN-09	Precisão decimal, representação monetária e arredondamento não estão definidos.	Escolher precisão ou regra de arredondamento altera o resultado de cálculos financeiros.	Definir escala decimal, moeda/representação e regra de arredondamento.
Permissões de ADMIN	UserRole; matriz de permissões	Há ações de ADMIN registradas, mas "autorização prévia" e limites de acesso não são detalhados; o modelo declara permissões pendentes.	Não é possível inferir o alcance de cada ação administrativa.	Validar permissões exatas, autorização prévia, escopo por recurso e restrição de pedidos em ambiente de teste.
Ciclo de vida do JWT	JWT; logout; armazenamento local	Token persistido por período indeterminado; duração/expiração, renovação e revogação de sessão não estão especificadas.	Definir tempo de sessão ou renovação altera comportamento de autenticação.	Confirmar política de expiração, renovação, revogação e efeito de logout.
Política de segurança	Autenticação e segurança	Política de senha, algoritmo/parâmetros de hash, limites de tentativas e proteção de dados pessoais não estão detalhados.	Não se pode inferir parâmetros de segurança que não foram aprovados.	Registrar os parâmetros e controles efetivamente adotados.
Acesso à rota de pedidos	API inicial — `GET /api/restaurants/:restaurantId/orders`	Acesso "Cliente proprietário" não está alinhado de forma clara à finalidade "Receber e aceitar pedido".	Corrigir o papel ou o método por suposição pode mudar o contrato.	Confirmar ator autorizado, finalidade e método da rota, preservando a rota até decisão.
Padronização de erros	Contratos de erro	Exemplo usa 422; tabela detalhada usa 400 para dados inválidos; `error.code` varia entre número HTTP e código semântico.	Clientes podem depender de convenções diferentes.	Definir status por cenário e formato de erro único, sem alterar os endpoints.
Mapeamento DTO/modelo	Contratos de API e modelo de dados	Exemplos de produto/pedido usam `situation`, `productId`,	Os exemplos podem ser apenas	Confirmar nomes e transformação entre DTOs e

Tema	Localização	Informação ausente ou conflitante	Por que não deve ser presumida	Decisão necessária
		`price`; modelo usa `status`, `restaurant_product_id`, `unit_price`.	representacionais; não há mapeamento definido.	entidades, mantendo os contratos aprovados.
Contrato de horários	Restaurant create/update	`opening_hours` é atributo obrigatório no modelo, mas foi deliberadamente adiado no contrato até a etapa 4.	O contrato não está completo para esse atributo enquanto a estrutura não for aprovada.	Definir `opening_hours` na etapa 4 e então atualizar request/response correspondentes, sem antecipar a estrutura.
Avaliações repetidas	Review	Unicidade de `order_item_id` está condicionada à decisão sobre avaliações repetidas.	Criar a restrição antes da decisão pode restringir o comportamento pretendido.	Confirmar se será permitida uma avaliação por item.
Integridade de categoria e produto	Category/RestaurantProduct	Regra exige mesma propriedade de restaurante; mecanismo de banco/constraint ainda não detalhado.	Não há definição física de índice/constraint ou estratégia de validação.	Na implementação física, registrar como a integridade será assegurada sem mudar a regra.
Contrato do carrinho	RN-09/carrinho	Recalcular total ao remover item está definido; endpoint e persistência do carrinho não estão descritos.	Criar rota/estrutura seria ampliar o contrato atual.	Documentar o mapeamento quando o contrato de carrinho for definido.
Dados pessoais editáveis	Matriz de permissões	A tabela inicial marca atualização de dados pessoais por cliente, responsável e administrador, enquanto a regra RN-04 limita a própria conta.	Não há contradição direta, mas não estão descritos campos ou operações de perfil.	Detalhar os campos editáveis e a autorização, preservando a regra de dados próprios.
Governança documental	Governança	Versão, histórico de alterações, responsável e status de aprovação não estão informados.	Não é possível inventar responsável ou status formal.	Definir os metadados de governança quando o projeto adotar esse controle.
Priorização	Priorização	Os RFs e RNs não têm prioridade individual documentada.	Não há ordem aprovada de implementação além das prioridades da lista de análise.	Definir prioridade dos requisitos, se necessária para o planejamento.
Estratégia de testes	Testes	Níveis de teste têm estrutura de diretórios e ferramentas citadas, mas não há critérios de entrada/saída ou cobertura.	Percentuais e gates não podem ser inventados.	Definir critérios de execução e aprovação da estratégia de testes.
Observabilidade e recuperação	Observabilidade e recuperação	Logs, métricas, tracing, backup e recuperação não têm requisitos operacionais definidos.	Não há parâmetros de retenção, frequência, RPO/RTO ou alertas aprovados.	Decidir quais controles serão exigidos e seus parâmetros.
Acessibilidade	Acessibilidade	Não há requisitos de acessibilidade explícitos.	Não é possível criar critérios de conformidade sem requisito aprovado.	Confirmar se haverá requisitos de acessibilidade e quais critérios serão utilizados.
Decisões técnicas finais	Arquitetura e tecnologias	A tabela de tecnologias declara opções como exemplos e não decisões definitivas; não há registro formal da decisão final por componente.	Tratar opções como tecnologias fechadas mudaria o estado de decisão.	Registrar decisões aprovadas e justificativas quando forem finalizadas.
Índices e constraints físicos	Banco de dados	Relacionamentos e algumas constraints são descritos, mas índices e estratégia completa de integridade física não estão especificados.	Índices e detalhes físicos dependem da implementação e não podem ser presumidos.	Registrar índices e constraints físicos quando o schema definitivo for elaborado.
Grafia de status	Estados do pedido	A documentação usa `CONCLUÍDO` e `CONCLUIDO` em locais diferentes.	A grafia pode afetar enum, persistência e contratos.	Confirmar a representação canônica do estado no código/API.

Tema	Localização	Informação ausente ou conflitante	Por que não deve ser presumida	Decisão necessária
Campo de situação do produto	Campos de produto	Exemplo de resposta usa `situation: active`; modelo utiliza `status`.	Não há confirmação se são nomes equivalentes no DTO ou se representam conceitos distintos.	Confirmar o nome e o mapeamento do atributo no contrato.

Rastreabilidade — requisitos, regras, dados, API e testes

A matriz abaixo utiliza apenas os identificadores existentes (`RF`, `RN` e `RNF`). "Não especificado" significa que o documento não permite estabelecer a ligação sem uma decisão ou detalhamento adicional.

Requisito	Regra de negócio relacionada	Entidades/estruturas	Endpoint/contrato relacionado	Critério de aceitação relacionado	Cenário de teste relacionado
RF-01	RN-04; e-mail único e conta por pessoa na seção `User`	`User`, `UserRole`	`POST /api/auth/register`; `POST /api/auth/login`; `POST /api/auth/logout`	Cadastro/login e formato de sucesso/erro descritos; campos completos pendentes.	Cadastro, login válido, credenciais inválidas, sessão/autenticação.
RF-02	RN-05	`Review`; `OrderItem`; `Order`; `User`	`POST /api/restaurants/:id/reviews`	Bloqueado para critério único até resolver escopo estabelecimento/produto.	Avaliação de estabelecimento/produto; vínculo com compra; execução final pendente.
RF-03	RN-01, RN-06, RN-07, RN-08, RN-09	`Order`, `OrderItem`, `Restaurant`, `RestaurantProduct`, `OrderEvent`	`POST /api/orders/create`; `GET /api/orders/`; `GET /api/restaurants/:restaurantId/orders`	Pedido de um restaurante, cliente autenticado, restaurante publicado/aberto, itens disponíveis, status `CRIADO`, snapshots e total.	Criação válida/inválida, estoque, horário, status, cancelamento pendente.
RF-04	RN-03 (autorização por restaurante)	`Restaurant`, `Order`, `OrderItem`, `Review`, `RestaurantProduct`	`GET /api/restaurant/dashboard`	Responsável autorizado consulta dados; métricas e formato detalhados pendentes.	Consulta autorizada e tentativa de acesso a outro restaurante.
RF-05	RN-02, RN-03, RN-06, RN-07	`ProductBase`, `RestaurantProduct`, `Category`, `OrderItem`	`POST /api/restaurants/:id/products`; `PATCH /api/products/:id`; `DELETE /api/products/:id`	CRUD de produto pelo responsável autorizado; preservação do histórico.	Criar/editar/excluir ou desativar; acesso não autorizado; produto referenciado em pedido.
RF-06	Conta única e papéis múltiplos; RN-04	`User`, `UserRole`	Rotas gerais de autenticação; não há rota separada documentada para atribuir papel.	Responsável usa conta com papel `RESPONSAVEL`; fluxo de atribuição de papel pendente.	Conta com papel adequado e tentativa de operação sem papel.
RF-07	Regras de restaurante; `publication_status` e `operating_status`	`User`, `UserRole`, `Restaurant`, `Order`	`POST /api/restaurants`; `GET /api/me/restaurants`; `GET /api/restaurants/{id}/management`; `PATCH /api/restaurants/{id}`; `PATCH /api/restaurants/{id}/publication`	Associação ao autenticado, CNPJ único, início `UNPUBLISHED`, gerenciamento do próprio restaurante e bloqueio de despublicação com pedidos ativos.	Criar, listar, consultar, editar, publicar, despublicar, acesso cruzado e validações.
RF-08	RN-06, RN-08; regras de estoque e preço	`RestaurantProduct`, `Order`, `OrderItem`, `Restaurant`	Rotas de produtos; `POST /api/orders/create`; fluxo de confirmação do pedido	Preço maior que zero, estoque não negativo, indisponibilidade com estoque zero, baixa apenas na confirmação e atomicidade.	Estoque insuficiente, concorrência, baixa e restauração do estoque.
RF-09	RN-02, RN-03, RN-07	`ProductBase`, `RestaurantProduct`, `Category`, `OrderItem`	`PATCH /api/products/:id`; operações de produto sob `/api/restaurants/:id/products`	Alterar preço, imagem, nome, descrição e categoria sem corromper snapshots.	Atualização autorizada, duplicidade de nome e preservação de pedido antigo.
RNF-01	—	Aplicação mobile/web, API e infraestrutura de dados	Abrange telas/rotas mais utilizadas; lista exata pendente.	Carga de 1.000 usuários simultâneos; limite de resposta não definido.	Teste de carga com K6; aprovação depende da métrica pendente.

Requisito	Regra de negócio relacionada	Entidades/estruturas	Endpoint/contrato relacionado	Critério de aceitação relacionado	Cenário de teste relacionado
RNF-02	—	Interface, API e camada de comunicação	Respostas de erro da API e comunicação Axios	Exibir mensagem compreensível e nunca indicar sucesso em falha.	Falha de conexão/503 e erro de servidor; política de retry pendente.

Rastreabilidade das entidades e restrições

Entidade/estrutura	Requisitos/regras relacionados	Restrições registradas	APIs relacionadas	Testes relacionados
`User` / `UserRole`	RF-01, RF-06, RN-03, RN-04	E-mail global único; chave composta `user_id` + `role`; múltiplos papéis por conta.	Autenticação; endpoints protegidos.	Cadastro/login, duplicidade de e-mail/papel, autorização por papel.
`Restaurant`	RF-07, RN-03, RN-08	Um responsável; CNPJ único; publicação separada do funcionamento.	Rotas de gerenciamento e rotas públicas.	Propriedade, publicação, horário, despublicação com pedidos ativos.
`ProductBase`	RF-05, RF-09, RN-02, RN-07	Definição compartilhável; regra de nome duplicado precisa de esclarecimento.	Rotas de produto; mapeamento ProductBase/RestaurantProduct pendente.	Duplicidade, atualização e preservação histórica.
`RestaurantProduct`	RF-05, RF-08, RF-09, RN-02, RN-03, RN-06, RN-07	Preço > 0; estoque >= 0; categoria do mesmo restaurante; status e histórico.	Rotas de catálogo/produto e criação de pedido.	Preço, estoque, categoria cruzada, exclusão/desativação.
`Category`	RF-05, RF-09	Categoria pertence a restaurante; produto usa categoria do mesmo restaurante.	Rotas de produto; endpoint específico de categoria não está listado.	Categoria de outro restaurante não pode ser associada.
`Order`	RF-03, RN-01, RN-08	Um restaurante; cliente com papel; ao menos um item; total soma subtotais; status inicial `CRIADO`.	Criar/listar pedidos e rota de pedidos do restaurante.	Autorização, restaurante único, status, total, criação.
`OrderItem`	RF-03, RF-08, RN-01, RN-06, RN-07	1–20 unidades; combinação order/product única; snapshots; subtotal consistente.	Criar pedido.	Limites, duplicidade de linha, snapshot, total e consistência.
`Review`	RF-02, RN-05	Produto adquirido; pedido concluído; nota 1–5; unicidade por item pendente.	`POST /api/restaurants/:id/reviews` (mapeamento pendente).	Propriedade, pedido concluído, produto adquirido, avaliação repetida.
`OrderEvent`	RF-03 e regras de transição	Cada mudança de status gera evento com ator, status anterior/novo e data.	Operações de mudança de status; rota específica não está listada.	Verificar evento após cada transição e consistência com pedido.

Governança, arquitetura e qualidade documental

Tema	O que está documentado	Lacuna sem decisão presumida
Arquitetura	Começar com monólito e evoluir para microsserviços conforme necessidade.	Critérios objetivos que determinam a evolução não estão definidos.
Tecnologias	React Native, React, Node/Express, REST, PostgreSQL/Prisma, Redis, JWT, Cloudinary, Postman, K6, Appium, GitHub e Actions aparecem como opções/exemplos.	A tabela afirma que não são recomendações definitivas; estado final de cada escolha deve ser registrado.
Organização do backend	Estrutura proposta por `config`, `modules`, `middlewares`, `lib`, `routes`, `tests` e `prisma`; camadas `controller`, `service`, `repository`, `schema` e `routes`.	A estrutura é uma proposta; não deve ser tratada como requisito de comportamento.
Testes	Ferramentas Postman, K6 e Appium; diretórios unitários, integração e fixtures.	Critérios formais de entrada/saída, cobertura e aprovação não definidos.
Disponibilidade	Não há meta explícita de disponibilidade/tempo ativo no documento.	Percentual de disponibilidade, janelas de manutenção e critérios de indisponibilidade não especificados.
Observabilidade	`OrderEvent` registra mudanças de status e ator.	Logs gerais, métricas, tracing, alertas, retenção e auditoria fora do fluxo de pedido não especificados.
Recuperação	Não há política registrada no documento.	Backup, periodicidade, retenção, recuperação e objetivos de recuperação não especificados.
Acessibilidade	Não há requisito explícito identificado.	Confirmar se haverá requisito e critérios de validação.

Tema	O que está documentado	Lacuna sem decisão presumida
Governança	Não há versão formal, histórico de alterações, responsável ou aprovação registrados.	Definir esses metadados se forem adotados pelo projeto.
Priorização	Lista de pontos da análise tem prioridades P0, P1 e P2.	Os requisitos RF/RN individuais não possuem prioridade formal na documentação-base.

Níveis de teste presentes na estrutura proposta

A organização de diretórios prevê `tests/unit`, `tests/integration` e `tests/fixtures`. As ferramentas listadas são Postman para testes de API, Grafana K6 para testes de carga e Appium para testes de interface. A documentação não define critérios de entrada/saída, cobertura mínima, política de regressão ou condições formais de aprovação; esses parâmetros permanecem pendentes.
